Finance Guide

Everything the Finance team does on PhysioGuidance — process payments, run the bank-transfer queue with malware gating, initiate and approve refunds, generate invoices, reconcile against gateways, run coupons and campaigns, and export the full audit trail.

PREPARED FOR
PhysioGuidance Finance Team

DATE
2026-08-17

DASHBOARD URL
app.physioguidance.org/en/dashboard/finance

PHYSIOGUIDANCE
Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Two entry points into Finance
2. The Finance sidebar at a glance
3. Overview dashboard
4. Payments queue
5. Bank Transfers queue — the full flow
6. Refunds — request, approve, reject, mark failed
7. Refund policy — the toggle set that governs eligibility
8. Invoices — generation, VAT, regenerate, cancel, verify
9. Reconciliation and Escalations
10. Discounts — Coupons, Campaigns, Course Sales
11. Reports — the eight report cards and exports
12. Settings — Refund Policy, Tax, VAT Treatments, Gateways
13. Audit Log
14. Support integrated into Finance
15. Admin-only super-actions available inside this UI
16. What Finance cannot do
17. Common questions
18. Where to look next

1. Two entry points into Finance

The same finance surface renders in two places, and it is important to know which one you are looking at.

1.1 Standalone Finance shell

Path: `/en/dashboard/finance/*`

This is what a user with the FINANCE role logs into. It has a dedicated `FinanceSidebar` and `FinanceNavbar`, so the visual chrome is tailored to money-only work. Every finance capability described in this guide is reachable from this shell.

1.2 Admin super-view

Path: `/en/dashboard/admin/finance/*` (mounted as `AdminFinanceShell`)

An ADMIN or SUPER_ADMIN opens the finance area from inside the admin dashboard. The very same components are re-mounted with an `embedded` prop and wrapped in `ActorRoleProvider role="admin"`. That provider is the trigger that reveals additional admin-only super-actions in the drawers — force status, replay webhook, sync from gateway, force-cancel invoice, force-approve refund, and the Escalations sub-routes that Finance can create but only Admin can claim or close.

Everything else is identical. Both shells share the same underlying REST endpoints, the same audit trail, and the same data.

---

2. The Finance sidebar at a glance

The Finance sidebar lists twelve items in this exact order:

#	Sidebar item	Path	Purpose
1	Overview	`/dashboard/finance`	KPI cards, revenue widgets, alerts rail
2	Payments	`/dashboard/finance/payments`	Every payment, with retry and cancel actions
3	Invoices	`/dashboard/finance/invoices`	Issued invoices with regenerate, resend, cancel
4	Refunds	`/dashboard/finance/refunds`	Request, approve, reject, mark-failed refunds
5	Bank Transfers	`/dashboard/finance/bank-transfers`	Receipt review queue with malware gating
6	Discounts	`/dashboard/finance/discounts`	Coupons, campaigns, course-level sales
7	Reports	`/dashboard/finance/reports`	Eight report cards with CSV, XLSX, PDF exports
8	Reconciliation	`/dashboard/finance/reconciliation`	Drift scans, findings, escalations
9	Audit Log	`/dashboard/finance/audit-log`	Immutable trail of every finance mutation
10	Support	`/dashboard/finance/support`	Finance-scoped support tickets
11	Looped-in	`/dashboard/finance/support/looped-in`	Tickets Support has looped Finance into
12	Settings	`/dashboard/finance/settings`	Refund policy, tax, VAT treatments, gateways

Every mutating action on any of these screens writes a row into the finance audit log. Actor IP is captured on every mutation for compliance evidence.

---

3. Overview dashboard

Path: `/dashboard/finance`

The overview is the daily starting point. It is composed of a set of widgets and a right-rail alert stack.

3.1 Widgets

- **Finance KPI cards** — headline numbers for today, this week, this month.
- **Revenue summary widget** — revenue by source and by period.
- **Finance analytics widget** — trend lines and comparators.
- **Recent transactions widget** — the last successful payments in reverse chronological order.
- **Refunds queue widget** — how many refund requests are waiting on approval.

3.2 Right-rail alerts

- **Financial alerts rail** — anything the reconciliation or webhook layer flagged.
- **Bank transfer rail** — receipts sitting in your queue with age.
- **Refunds queue rail** — refunds awaiting your decision.
- **Active coupons rail** — coupons currently redeemable, with usage counts.
- **Reconciliation drifts rail** — open findings that need triage.

3.3 Backing endpoints

The overview reads from a dedicated read-only set of endpoints so it renders fast without touching the write path.

Endpoint	What it returns
`GET /finance/summary`	The KPI card values
`GET /finance/revenue-by-source`	Slice by gateway
`GET /finance/payment-status`	Distribution of statuses
`GET /finance/refund-rate`	Refund ratio for the period
`GET /finance/invoice-aging`	Aging buckets for outstanding invoices
`GET /finance/bank-transfer-review-aging`	Age distribution of pending receipts
`GET /finance/revenue-trend`	Time-series revenue
`GET /finance/top-courses`	Top-selling courses
`GET /finance/revenue-by-course`	Per-course revenue
`GET /finance/revenue-by-instructor`	Per-instructor revenue
`GET /finance/revenue-by-delivery-type`	Split across delivery types
`GET /finance/chargeback-rate`	Chargeback ratio for the period

Everything on the overview is a read. Nothing on this screen mutates state, and nothing here writes audit rows.

---

4. Payments queue

Path: `/dashboard/finance/payments`

This is the master list of every payment ever created — pending, awaiting approval, paid, failed, refund in progress, or refunded.

4.1 List and filter

`GET /payments` returns a paginated list. Available filters:

- Status
- Gateway (Moyasar, Tamara, Tabby, Bank Transfer)
- Payment method

- Free-text search across buyer, course, and payment ID
- Date range
- `includeAbandoned` toggle

4.2 Payment detail drawer

Click any row to open the drawer, which calls:

- `GET /payments/:paymentId/detail` — the payment with a per-line-item breakdown (each course line, amounts, discounts, VAT snapshot, gateway response snapshot).
- `GET /payments/:paymentId/audit` — the audit trail for this specific payment, in reverse chronological order.

4.3 Row-level actions Finance can take

Action	Endpoint	Notes
View detail	`GET /payments/:paymentId/detail`	Read-only
Export	via reports	Filter set exported as CSV/XLSX/PDF
Retry Tamara capture	`POST /finance/tamara/retry/:paymentId`	Re-authorises the capture call to Tamara
Cancel pre-capture Tamara order	`POST /finance/tamara/cancel/:paymentId`	Requires `reason` at least 5 characters
Trigger refund from row	`POST /refunds`	For Finance role, backend silently reroutes into `adminInitiateRefund` with `bypassPolicy=true`; audited

4.4 Abandoned sweep

Finance can run a manual sweep to clean up abandoned payments:

- `POST /payments/abandoned-sweep/run` — triggers the same job the cron runs. Throttled to 6 calls per minute so it cannot be hammered by accident.

4.5 Gateway health

- `GET /payments/gateways/health` — quick liveness check across all connected gateways. Use it when a payment fails and you want to know whether the vendor is down before contacting the customer.

4.6 Admin-only super-actions on a payment

When the drawer is opened inside the admin super-view, three extra buttons appear. All three require a `reason` at least 5 characters and are audit-logged.

Super-action	Endpoint	Effect
Force status	`POST /admin/finance/payments/:id/force-status`	Flip a payment to paid or failed manually
Replay webhook	`POST /admin/finance/payments/:id/replay-webhook`	Re-run the success path as if the webhook had just arrived
Sync from gateway	`POST /admin/finance/payments/:id/sync-from-gateway`	Poll live gateway status and apply it

These do not show up in the Finance-role shell. They appear only when `ActorRoleProvider role="admin"` is present, which is exclusively inside `AdminFinanceShell`.

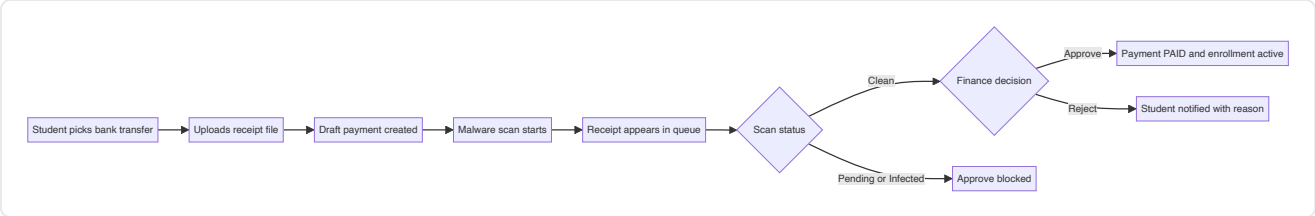
---

5. Bank Transfers queue — the full flow

Path: `/dashboard/finance/bank-transfers`

Bank transfer is the only payment method that requires a human in the loop. A student uploads a proof-of-transfer receipt, an antivirus scanner runs, and Finance approves or rejects.

5.1 End-to-end flow



5.2 Step 1 — Student submits a draft

`POST /payments/bank-transfer/submit-draft` (multipart)

The student uploads the receipt file along with the list of course IDs being purchased and an optional discount code. The service does everything in a single transaction:

- Creates the payment record with status `AWAITING_APPROVAL`.
- Creates one payment item per course line.
- Creates the receipt record with `scanStatus=PENDING` and `reviewStatus=pending`.
- Uploads the file to the private bucket.

The student can immediately download a proforma PDF for their records.

5.3 Step 2 — Automatic malware scan

An Eventarc-triggered ClamAV scanner watches the `physioguidance-private` bucket. As soon as a new object lands, the scanner:

1. Downloads and inspects the file.
2. Writes back a metadata verdict.
3. A backend poll flips the receipt's `scanStatus` to `clean`, `infected`, or leaves it `pending` if the scan has not finished.

5.4 Step 3 — Queue endpoints Finance uses

Endpoint	Purpose
`GET /payments/bank-transfer/queue/list`	List receipts, filter by pending / approved / rejected / all, plus search
`GET /payments/bank-transfer/queue/summary`	KPIs: `pendingCount`, `pendingAmount`, `scanPendingCount`, `oldestPendingHours`, `infectedCount`
`GET /payments/bank-transfer/queue/receipts/:receiptId`	Drawer detail with refund-info snapshot, short-lived signed download URL, student and course context
`GET /payments/bank-transfer/queue/receipts/:receiptId/replacement-chain`	Every receipt ever uploaded for that payment, oldest to newest
`GET /payments/bank-transfer/queue/receipts/:receiptId/audit`	Unified timeline of scan events, reviews, notes, approvals

5.5 Step 4 — Downloading the receipt

`GET /payments/bank-transfer/receipts/:receiptId/download`

Returns a short-lived signed URL. The service refuses to hand out a URL when the scan status is not clean:

- HTTP **403** when the file is `infected` (quarantined)
- HTTP **425** when the scan is still `pending` (too early)

This gate is enforced in the service layer, not just the UI. Even a direct API call cannot bypass it.

5.6 Step 5 — Approving a receipt

`PATCH /payments/bank-transfer/queue/receipts/:receiptId/approve`

Optional body: `internalNote` (visible only to Finance and Admin).

The service refuses to approve when scan status is `INFECTED` or `PENDING`. On a successful approve, the following happen atomically:

- Payment status flips to PAID.
- Enrollment is activated for every course line.
- Invoice PDFs are generated in both locales with the correct VAT treatment per line.
- Confirmation email is sent to the buyer.
- Audit log row `bank_transfer.approved` is written with your actor identity and IP.

5.7 Step 6 — Rejecting a receipt

`PATCH /payments/bank-transfer/queue/receipts/:receiptId/reject`

Body:

- `reviewerNote` (required, at least 5 characters) — this text is shown to the student verbatim in the rejection email, so it must be clear and actionable.
- `internalNote` (optional) — stays private to Finance and Admin.

On reject:

- Payment rolls back to a pending state so the student can re-upload a corrected receipt without starting a new order.
- Rejection email is sent to the student with your `reviewerNote`.
- Audit log row `bank_transfer.rejected` is written.

5.8 Bank details shown to students

Students are shown the bank name, beneficiary, IBAN, SWIFT, and instructions in both languages. These come from `system_settings.payments.bankTransfer` and are exposed by `GET /payments/bank-transfer/details`. Admin edits them in the platform settings screen.

---

6. Refunds — request, approve, reject, mark failed

Path: `/dashboard/finance/refunds`

The Refunds screen is where Finance both initiates refunds on behalf of buyers and processes refunds that students requested through their own dashboard.

6.1 List, summary, and lookups

Endpoint	Purpose
`GET /refunds`	Paginated list with filters
`GET /refunds/summary`	KPI counts by state
`GET /refunds/refundable-payments`	Search-and-suggest paid payments that are eligible for refund
`GET /refunds/preview/:paymentId`	Pre-computes gateway deduction and partial refund math before you commit
`GET /refunds/:id`	A refund record
`GET /refunds/:id/detail`	Full detail with related payment, invoice, and enrollments
`GET /refunds/:id/audit`	Audit trail for this refund
`GET /refunds/export`	CSV export of the current filter set

6.2 Initiating a refund

`POST /refunds`

The backend inspects the caller's active role:

- **Student** — subject to the student refund policy (see section 7).
- **Finance, Admin, Super Admin** — silently rerouted into `adminInitiateRefund` with `bypassPolicy: true`. Finance can therefore initiate a refund on any paid payment without the eligibility gates blocking it.

Alternatively, use the explicit administrator entry point directly:

`POST /refunds/admin-initiate` — same behaviour, made explicit for clarity in audit logs.

6.3 Approving a queued student refund

`PATCH /refunds/:id/approve`

Body:

- `approvalNote` (optional)
- `approvedAmount` (optional; use it to approve a partial refund even if the student asked for the full amount)

6.4 Rejecting a refund

`PATCH /refunds/:id/reject`

Body:

- `reason` (required, at least 5 characters)

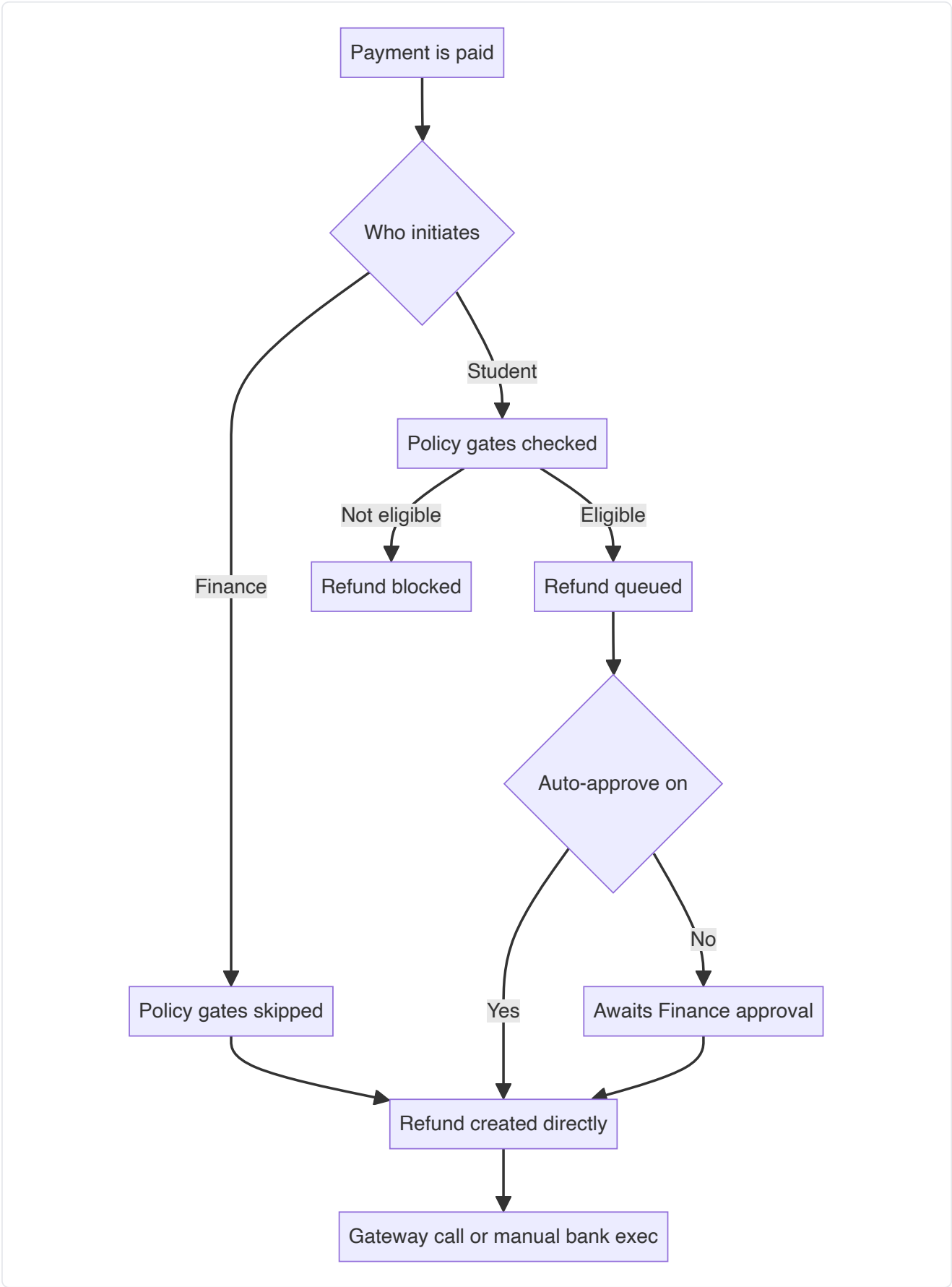
The reason is included in the rejection notification to the student and stored in the audit trail.

6.5 Marking a refund as failed

`PATCH /refunds/:id/mark-failed`

Use this when the gateway returned an error the platform cannot automatically recover from, or when a manual bank refund could not be executed. This records the write-off with the reason and stops the refund from being retried.

6.6 Flow diagram



7. Refund policy — the toggle set that governs eligibility

The refund policy lives in `system_settings.refund_policy` and is edited from **Settings** → **Refund Policy** (section 12.1). The same rules that block a student-initiated refund also drive the auto-approve behaviour.

7.1 Eligibility toggles

Toggle	What it blocks
`blockAfterCourseStart`	Refund requests after the course start date. Self-paced courses skip this check.
`blockAfterAttendance`	Refund requests once the student has recorded attendance.
`blockAfterCertificate`	Refund requests once a certificate has been issued.
`blockAfterCmeGranted`	Refund requests once CME hours have been granted to the student.

Toggle any of these off to loosen the policy. All four default to on.

7.2 Auto-approve

`autoApproveEligible` — when on, a student refund request that passes every eligibility gate is auto-approved without waiting on Finance action. Leave it off to keep every refund on Finance's queue for review.

7.3 Per-gateway deduction and partial refund capability

Each gateway has two per-gateway settings that snapshot into every refund preview.

Gateway	`deductionPercent` default	`partialRefundCapable` default
Moyasar	0%	On
Tamara	10%	On
Tabby	10%	On
Bank Transfer	0%	On

Deduction models the fee the gateway keeps on a refunded transaction. Finance is free to adjust these values to match the current contract with each vendor.

7.4 Finance-initiated refunds bypass all of the above

When a Finance actor initiates a refund, the backend routes into `adminInitiateRefund` with `bypassPolicy: true` and skips every eligibility gate. This is deliberate — Finance is trusted to override policy on legitimate grounds, and every override is captured in the audit log with the actor identity, IP, and reason.

7.5 Admin-only force approve

`POST /admin/finance/refunds/:id/force-approve`

There is one case even Finance cannot resolve unaided — a refund that would violate a lifecycle policy the platform enforces regardless of role (for example, a certificate has already been issued and CME hours have already been granted). Admin can force-approve such a refund. The reason is required, must be at least 5 characters, and the resulting audit row is `refund_approve_admin_override` with the internal note prefixed `[ADMIN POLICY BYPASS]`.

8. Invoices — generation, VAT, regenerate, cancel, verify

Path: `/dashboard/finance/invoices`

Every successful payment produces an invoice. The Invoices screen is the archive plus a set of maintenance actions.

8.1 List and lookups

Endpoint	Purpose
`GET /invoices`	Paginated list with filters
`GET /invoices/summary`	KPI counts and totals
`GET /invoices/export.csv`	Bulk export of the current filter set
`GET /invoices/:id`	Invoice record
`GET /invoices/:id/detail`	Full detail with buyer, courses, VAT lines
`GET /invoices/:id/audit`	Audit trail for this invoice
`GET /invoices/:id/pdf?locale=en`	Signed URL to the English PDF
`GET /invoices/:id/pdf?locale=ar`	Signed URL to the Arabic PDF

8.2 Maintenance actions

Endpoint	Purpose
`POST /invoices/:id/regenerate-pdf?locale=en`	Re-render the English PDF using current settings (updated tax number, company name, footer)
`POST /invoices/:id/regenerate-pdf?locale=ar`	Same, Arabic
`POST /invoices/:id/cancel`	Cancel a non-paid invoice; `reason` at least 4 characters
`POST /invoices/:id/resend`	Re-email the invoice to the buyer
`POST /invoices/overdue-sweep/run`	Manual trigger for the overdue sweep cron

8.3 Public verification

`GET /invoices/verify/:invoiceNumber`

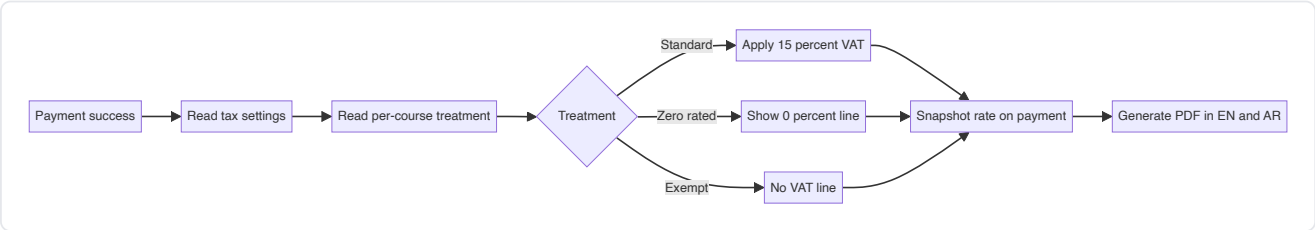
A QR code printed on every invoice PDF resolves to this public endpoint. It confirms the invoice number belongs to a real, non-cancelled record and returns a small verification payload — buyer name, amount, date, ZATCA tax number. Auditors and buyers can verify authenticity without logging in.

8.4 Invoice content and structure

- **Number format** — `INV-{year}-{5-digit sequence}` (for example `INV-2026-00042`). Sequence is per-year and monotonic.
- **Storage keys** — bilingual PDFs are stored at `invoices/{invoiceNumber}-{locale}.pdf` in the private bucket.

- **Content fields** — invoiceNumber, subtotal, taxAmount, totalAmount, currency `SAR`, buyer, course title in both English and Arabic, gateway, paymentMethod, issued date, paid date, itemCount, ZATCA tax number and company name from tax settings, and a bilingual footer.

8.5 VAT computation flow



The rate and treatment are snapshotted on the payment metadata at the moment of computation, so historical invoices never change if the global rate is later adjusted.

8.6 Admin-only super-actions on invoices

Inside the admin super-view, three additional buttons show up on an invoice. All three require a `reason` at least 5 characters.

Super-action	Endpoint	Effect
Force cancel	`POST /admin/finance/invoices/:id/force-cancel`	Can cancel even a PAID invoice; the internal note is prefixed `[ADMIN OVERRIDE]`
Resend email	`POST /admin/finance/invoices/:id/resend-email`	Sends the invoice email again on behalf of admin
Regenerate PDF	`POST /admin/finance/invoices/:id/regenerate-pdf`	Regenerates the PDF as an admin action

---

9. Reconciliation and Escalations

Path: `/dashboard/finance/reconciliation`

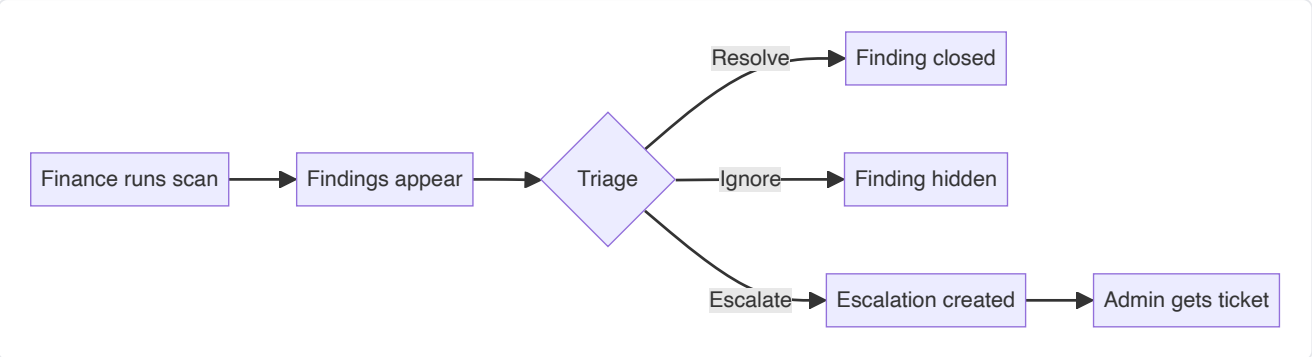
Reconciliation compares the platform's payment records against each gateway and surfaces drifts as findings. Every finding is either resolved, ignored, or escalated to Admin for a heavier intervention.

9.1 Scan and findings endpoints

Access: FINANCE, ADMIN, SUPER_ADMIN.

Endpoint	Purpose
`GET /payments/reconciliation/stats`	Counts and aging for the finding queue
`GET /payments/reconciliation/findings`	List findings with filters
`GET /payments/reconciliation/findings/export`	CSV, XLSX, or PDF export; the export itself is audit-logged
`POST /payments/reconciliation/scan`	Run an on-demand scan. `gateway=moyasar|tamara|tabby|bank_transfer|all`. Default window is last 7 days. Throttled to 5 calls per minute
`PATCH /payments/reconciliation/findings/:id/resolve`	Mark one finding resolved
`PATCH /payments/reconciliation/findings/:id/ignore`	Mark one finding ignored
`POST /payments/reconciliation/findings/bulk-resolve`	Bulk resolve with a shared note
`POST /payments/reconciliation/findings/bulk-ignore`	Bulk ignore with a shared note

9.2 Finding triage flow

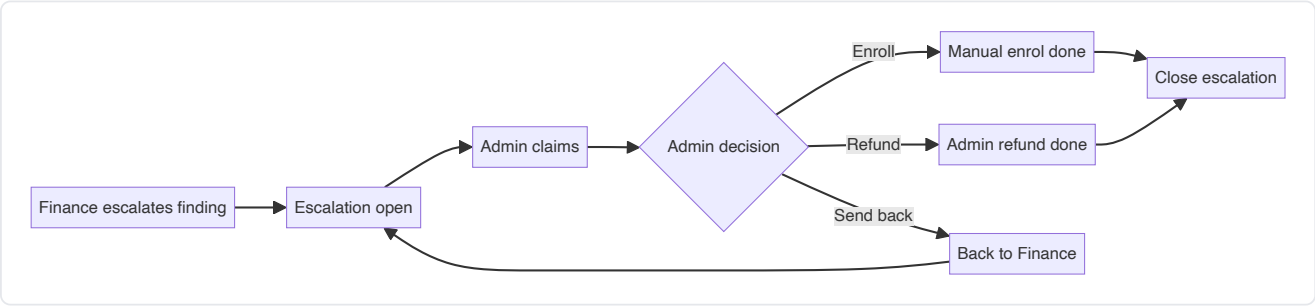


9.3 Escalations

Finance can create an escalation from any finding but cannot claim, close, enroll, or refund from one. That is where Admin takes over.

Endpoint	Who	Purpose
`POST /payments/reconciliation/findings/:findingId/escalate`	FINANCE (throttle 10 per minute)	Create an escalation with `note`, `proposedAction`, `urgency`
`GET /payments/reconciliation/escalations`	FINANCE, ADMIN	List escalations
`GET /payments/reconciliation/escalations/stats`	FINANCE, ADMIN	KPI counts
`GET /payments/reconciliation/escalations/:id`	FINANCE, ADMIN	Full detail with linked finding and history
`PATCH /payments/reconciliation/escalations/:id/claim`	ADMIN only	Admin takes ownership
`PATCH /payments/reconciliation/escalations/:id/send-back`	ADMIN only	Send back to Finance with a note
`PATCH /payments/reconciliation/escalations/:id/close`	ADMIN only	Close with resolution note
`POST /payments/reconciliation/escalations/:id/enroll`	ADMIN only	Manually enrol the student in the missing course
`POST /payments/reconciliation/escalations/:id/refund`	ADMIN only	Admin-initiated refund from the escalation; stays APPROVED on gateway failure for retry

9.4 Escalation lifecycle



10. Discounts — Coupons, Campaigns, Course Sales

Path: `/dashboard/finance/discounts`

The Discounts area has five tabs: **Overview**, **Codes**, **Early-Bird**, **Campaigns**, and **Course Sales**.

10.1 Coupons — codes and early-bird

Endpoint	Purpose
`GET /discounts`	List with filters: status, type, autoApply, earlyBird, campaignId, scope
`POST /discounts`	Create a coupon
`GET /discounts/:id`	Coupon record
`PATCH /discounts/:id`	Update
`DELETE /discounts/:id`	Delete
`GET /discounts/:id/audit`	Coupon audit trail
`POST /discounts/bulk`	Bulk-generate coupon codes for a campaign or promotion
`GET /discounts/early-bird`	Early-bird coupons only
`GET /discounts/batches`	Bulk-generation batch history
`GET /discounts/summary`	Aggregate metrics
`GET /discounts/chart-series`	Time-series for the overview
`GET /discounts/analytics`	Redemption analytics
`GET /discounts/activity/recent`	Recent redemptions
`GET /discounts/export`	Export the current filter set

10.2 Campaigns

Endpoint	Purpose
`GET /discounts/campaigns`	List with filters: status, search, `campaignType=standard|early_bird`
`POST /discounts/campaigns`	Create
`GET /discounts/campaigns/:id`	Detail
`PATCH /discounts/campaigns/:id`	Update
`DELETE /discounts/campaigns/:id`	Delete
`GET /discounts/campaigns/report`	Performance report

Fields on a campaign:

- `name`, `nameAr`, `description`
- `startsAt`, `endsAt`
- `budget`
- `showOnSite`, `bannerCta`
- `campaignType` (`standard` or `early_bird`)
- `discountType` (`none`, `percentage`, or `fixed`)
- `discountValue`
- `scope` (`code_only`, `all_paid`, `by_delivery_type`, or `specific_courses`)
- `scopeDeliveryTypes` — list of delivery types when scope is delivery-based

- `scopeCourseIds` — list of course IDs when scope is specific-courses
- `allowsCoupon` — whether coupons can stack on top of this campaign

10.3 Course-level sales

Endpoint	Purpose
`GET /discounts/course-sales`	List of active course sales
`PUT /discounts/course-sales/:courseId`	Set or update the sale price for a course

Interaction rule: if `sale.onSale` is true and `sale.saleAllowsCoupon` is false, coupons are rejected on top of the sale. That way, Finance can run either a sale or a coupon promotion without accidentally stacking two discounts on the same course.

10.4 Public storefront endpoints

Two endpoints are exposed unauthenticated for use on the storefront:

- `POST /discounts/auto-apply` — computes the best early-bird for the current cart.
- `GET /discounts/active-banner-campaign` — returns the campaign to render on the storefront banner right now.

---

11. Reports — the eight report cards and exports

Path: `/dashboard/finance/reports`

The Reports screen shows eight report cards. Each card has a JSON preview endpoint for on-screen display and an export path that produces CSV, XLSX, or PDF.

11.1 The eight report cards

Report	Preview endpoint
Revenue	`GET /finance/reports/revenue`
Refunds	`GET /finance/reports/refunds`
Payments	`GET /finance/reports/payments`
Course Sales	`GET /finance/reports/course-sales`
Webinar Sales	`GET /finance/reports/webinar-sales`
Student Purchases	`GET /finance/reports/student-purchases`
Finance Activity	`GET /finance/reports/finance-activity`
VAT	`GET /finance/reports/vat`

11.2 Universal export endpoint

`GET /finance/export?reportType=<key>&format=csv|xlsx|pdf&startDate=&endDate=&<filters>`

Supported filters vary by report type but include: `gateway`, `method`, `status`, `reason`, `courseId`, `sessionId`, `deliveryType`, `search`, `top`, `actorId`, `entityType`, `action`.

Every call to `/finance/export` writes an audit-log row `report_exported` including the filter set used, so you have an exact record of which report was pulled by which actor with what filters.

11.3 Legacy payments export

`GET /finance/export?reportType=payments-legacy&format=csv|pdf`

The legacy payments export is still supported for CSV and PDF via the historical `exportPaymentsCsv` and `exportPaymentsPdf` paths, for compatibility with existing accounting workflows.

11.4 Focused exports elsewhere in the UI

Individual screens also expose focused CSV exports so you do not have to jump to the Reports screen to pull a quick file:

Screen	Endpoint
Refunds	`GET /refunds/export`
Invoices	`GET /invoices/export.csv`
Discounts	`GET /discounts/export`
Reconciliation findings	`GET /payments/reconciliation/findings/export` (CSV, XLSX, or PDF)

---

12. Settings — Refund Policy, Tax, VAT Treatments, Gateways

Path: `/dashboard/finance/settings`

Four tabs.

12.1 Refund Policy tab

Endpoint	Purpose
`GET /finance/settings/refund-policy`	Read (cached 30 seconds)
`PUT /finance/settings/refund-policy`	Update

Editable fields:

- `blockAfterCourseStart`, `blockAfterAttendance`, `blockAfterCertificate`, `blockAfterCmeGranted`
- `autoApproveEligible`
- Per-gateway `deductionPercent` and `partialRefundCapable` for moyasar, tamara, tabby, bank_transfer

Every save is audit-logged.

12.2 Tax tab

Endpoint	Purpose
`GET /finance/settings/tax`	Read
`PUT /finance/settings/tax`	Update

Persisted fields:

- Global VAT rate (default **15**)
- ZATCA tax number
- Company name (English)
- Company name (Arabic)
- Default VAT treatment (`standard` , `exempt` , or `zero_rated`)

The rate is snapshotted on every payment at the moment of computation, so historical invoices are stable across rate changes.

12.3 VAT Treatments tab

Deep-link path: `/dashboard/finance/settings/tax-treatments`

Per-course VAT treatment is edited here. Some courses (for example medical training exempt from Saudi VAT) need a treatment other than the platform default.

Endpoint	Purpose
`GET /finance/settings/vat-treatments`	Search, filter by treatment, paginated. Also includes a 90-day sales count per course for prioritisation.
`PUT /finance/settings/vat-treatments/:courseId`	Set the treatment for a single course. Value ∈ `standard` , `exempt` , `zero_rated` .
`POST /finance/settings/vat-treatments/bulk`	Bulk update many courses in one call.

All three write audit rows.

12.4 Gateways tab

`GET /finance/settings/gateways`

Read-only for Finance. Returns a sanitised projection of `system_settings.payments` — enough to see how each gateway is currently configured, without exposing raw secret values.

Fields returned:

- `activeGateway`
- `testMode` (true or false)
- `currency`
- Per-gateway `enabled` flag
- `publishableKey`
- `hasSecret` (boolean)
- `secretMasked` (last four characters only)

- Bilingual `bankTransfer` block
- `invoiceFooter`
- `autoSendReceipt`

Toggles and secrets are written only by ADMIN. The gateway edit surface lives in `/dashboard/admin/settings`. Inside `AdminFinanceShell`, the Gateways tab surfaces the same read-only projection plus a clear call-out link into the admin settings screen for edits.

Admin also has three additional endpoints on this tab:

Endpoint	Purpose
`GET /admin/finance/gateways/runtime-status`	Live probe of every gateway
`POST /admin/finance/gateways/:gateway/test`	Test-fire a call to `moyasar`, `tamara`, or `tabby` to verify credentials work
`POST /admin/finance/gateways/refresh-cache`	Force the settings cache to refresh across all Cloud Run instances

---

13. Audit Log

Path: `/dashboard/finance/audit-log`

Every mutating finance endpoint writes a row. The Audit Log tab is where Finance reads that trail.

Endpoint	Purpose
`GET /audit-logs/finance`	Filtered list
`GET /audit-logs/finance/filters`	Filter option lists (actors, entity types, actions, date ranges)
`GET /audit-logs/finance/actors`	Actor picker
`GET /audit-logs/finance/summary`	Top counts for the overview cards
`GET /audit-logs/finance/export`	CSV, XLSX, or PDF export

What gets logged:

- Every payment mutation (create, force-status, replay-webhook, sync-from-gateway, retry-capture, cancel).
- Every refund mutation (initiate, approve, reject, mark-failed, force-approve).
- Every invoice mutation (regenerate, cancel, resend, force-cancel).
- Every receipt review (approve, reject, note).
- Every policy change (refund policy, tax, VAT treatments).
- Every gateway settings change (writes attributed to Admin).
- Every escalation transition (create, claim, close, send-back, enrol, refund).
- Every report export (`report_exported` with the filter set).

The audit trail is append-only. There is no delete or update path exposed anywhere in the platform.

---

14. Support integrated into Finance

Path: `/dashboard/finance/support`

Support is embedded in the Finance shell so payment, invoice, refund, and integration questions can be handled without leaving the money workspace.

14.1 Support tab

`FinanceSupportContent.tsx` renders the finance-scoped tickets. Finance can:

- Submit a new ticket categorised by area (payments, invoices, refunds, integrations).
- Track tickets Finance has opened.
- Reply to tickets Finance owns.

14.2 Looped-in tab

Path: `/dashboard/finance/support/looped-in`

This is where Support has looped Finance in for input on a ticket owned by the Support team. It is a lightweight collaborator view — Finance can read and reply but does not take ownership.

14.3 Individual looped-in ticket

Path: `/dashboard/finance/support/:id`

The `CollaboratorTicketView` component renders the specific ticket with the full thread and a reply box.

---

15. Admin-only super-actions available inside this UI

The following endpoints are visible to Finance in the abstract — Finance sees them in the drawer when opened inside the admin super-view — but only ADMIN and SUPER_ADMIN can call them. Every one requires a `reason` of at least 5 characters and every call is audit-logged.

Endpoint	What it does
`POST /admin/finance/payments/:id/force-status`	Flip a payment to paid or failed manually
`POST /admin/finance/payments/:id/replay-webhook`	Re-run the success path as if the webhook had just arrived
`POST /admin/finance/payments/:id/sync-from-gateway`	Poll the live gateway and apply the returned status
`POST /admin/finance/refunds/:id/force-approve`	Approve a refund that lifecycle policy would otherwise block; internal note is prefixed `[ADMIN POLICY BYPASS]`
`POST /admin/finance/invoices/:id/force-cancel`	Cancel even a PAID invoice; internal note is prefixed `[ADMIN OVERRIDE]`
`POST /admin/finance/invoices/:id/resend-email`	Resend the invoice email as an admin action
`POST /admin/finance/invoices/:id/regenerate-pdf`	Regenerate the PDF as an admin action
`PATCH /payments/reconciliation/escalations/:id/claim`	Take ownership of an escalation
`PATCH /payments/reconciliation/escalations/:id/send-back`	Return an escalation to Finance
`PATCH /payments/reconciliation/escalations/:id/close`	Close an escalation
`POST /payments/reconciliation/escalations/:id/enroll`	Manually enrol the student from the escalation
`POST /payments/reconciliation/escalations/:id/refund`	Admin-initiated refund from the escalation
`GET /admin/finance/gateways/runtime-status`	Live gateway health
`POST /admin/finance/gateways/:gateway/test`	Test-fire against a specific gateway
`POST /admin/finance/gateways/refresh-cache`	Refresh the gateway settings cache platform-wide

`ActorRoleProvider role="admin"` is the trigger that reveals these buttons in the drawer. When Finance opens the same drawer from the standalone shell, these buttons are simply not rendered.

---

16. What Finance cannot do

Finance is a scoped role. Everything below sits outside the Finance surface by design.

- Create, edit, suspend, or delete user accounts.
- Change user roles or grant admin privileges.
- Create, edit, publish, or archive courses.
- Edit course refund windows at course creation time.
- Write gateway secrets or toggle gateway on and off (Admin only, in `/dashboard/admin/settings`).
- Claim, close, enrol from, or refund from a reconciliation escalation (Admin only — Finance creates escalations and hands them off).

- Force-approve a refund that lifecycle policy blocks (Admin only via `force-approve`).
- Force-cancel a paid invoice (Admin only via `force-cancel`).
- Force-flip a payment status, replay a webhook, or sync a payment from the gateway (Admin only).
- Approve a bank-transfer receipt whose malware scan is pending or infected — this is a hard technical gate at the service layer; no role can bypass it.
- Manually edit or delete audit-log rows — the audit trail is append-only.

Two things are worth calling out:

1. **The bank-transfer malware gate is the only workflow with a hard technical gate Finance cannot bypass.** Every other gate in the platform is a policy gate that Finance-initiated refunds override. Infected or still-pending receipts are refused at the service layer with 403 or 425 responses, no exceptions.
2. **New payments and payment failures reach Finance automatically.** The notified-roles filter for both `payment_success` and `payment_failed` events includes FINANCE, so the Finance inbox lights up as things happen without anyone needing to configure a subscription.

---

17. Common questions

A student uploaded a bank-transfer receipt and I do not see it in my queue. Refresh. If it is still missing, the scan is likely still running — very fresh uploads may take a few seconds before appearing. Confirm the payment exists in the Payments queue filtered to `awaiting_approval`, then re-check the Bank Transfers queue.

The approve button on a receipt is disabled and the file looks fine. Check the scan status chip in the drawer. It will read Pending (scanner is still working) or Infected (scanner flagged the file). If Infected, reject the receipt with a clear reviewer note asking the student to re-upload from a clean source — that reason is emailed verbatim to the student.

A refund I approved is stuck in processing for hours. For Moyasar, Tabby, or Tamara, the reconciliation cron re-checks gateway status regularly. If a refund stays in processing beyond half an hour, open the payment detail and run a sync-from-gateway (Admin) or look at the reconciliation findings — the gateway may have already responded and the drift shows up as a finding.

A student says they were charged but no enrollment appeared. Open the payment detail. If the payment record is `pending` or `failed`, ask Admin to run sync-from-gateway. If the gateway confirms paid, the sync will fix the enrolment. If the gateway confirms failed, the customer was not actually charged and can retry safely.

Can I initiate a refund even if the course refund window has closed? Yes. When Finance initiates a refund, the backend routes into `adminInitiateRefund` with `bypassPolicy: true` and skips every eligibility gate. Only lifecycle policy that blocks even Admin (certificate already issued, CME already granted) requires Admin `force-approve`.

Where can I run a report on VAT collected in a period? Reports → VAT card, or `GET /finance/export?reportType=vat&format=xlsx&startDate=…&endDate=…`. The export is audit-logged.

Can I approve a bank-transfer receipt on behalf of another Finance user? Yes. Any Finance user can approve any receipt in the queue. The approval is stamped with your actor identity and IP.

How do I change VAT for a specific course that should be exempt? Settings → VAT Treatments (deep link `/dashboard/finance/settings/tax-treatments`). Search for the course, set the treatment to `exempt` or `zero_rated`, save. New invoices for that course use the new treatment immediately; historical invoices are not touched because the rate and treatment are snapshotted at the moment of payment.

A campaign scope should include only my premium delivery types. How do I set that? Campaigns tab → create or edit → set `scope=by_delivery_type` and populate `scopeDeliveryTypes` with the list of qualifying delivery types.

Coupons and course sales — can they stack? Only if the course sale is configured with `saleAllowsCoupon=true`. When it is false, the backend rejects the coupon at redemption on a course that is on sale.

How do I run a one-off reconciliation scan against Tamara only? Reconciliation tab → Scan button, choose Tamara. Or `POST /payments/reconciliation/scan` with `gateway=tamara`. Throttle is 5 calls per minute.

A reconciliation finding needs an actual refund or manual enrolment — what do I do? Escalate it. Finance cannot enrol from a finding or issue a refund tied to one. Click Escalate on the finding, choose the proposed action and urgency, and hand it to Admin. Admin claims, resolves, and closes.

A gateway is down and I want to prevent new payments through it right now. Ask Admin to disable the gateway in `/dashboard/admin/settings` — Finance sees the read-only gateway view but cannot toggle. Once Admin flips it off, the platform stops offering that gateway to buyers.

Where do I see every action a specific Finance teammate took last month? Audit Log → Actor filter, pick the teammate, set the date range, export as XLSX.

Can historic invoices be edited if the tax rate changes? No. Invoices are immutable snapshots. A rate change updates future invoices only. Regenerating a PDF renders the same underlying data with any settings changes to layout — the amounts do not change.

---

18. Where to look next

- **How admins run the platform end to end** — admin-guide.md
- **What students see when they pay** — student-guide.md
- **The malware gate on receipt uploads** — malware-unscanned-file-gate.md
- **Full REST API reference for every endpoint used here** — api-reference.md
- **Environment secrets, gateway credentials, and rotation policy** — environment-configuration-CONFIDENTIAL.md
- **Monitoring and alerting on payment failures** — monitoring-alerting.md